

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Gayathri .B.R	Reg.No.:	192521330
Department:	IT	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I _Gayathri B.R(192521330)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1. DMA-Based Data Transfer Algorithm

Objective: Transfer data from an I/O device to memory while using interrupts and minimizing CPU involvement.

PSEUDOCODE: DMA_BASED_TRANSFER

START

Initialize DMA Controller

Initialize I/O Device

Initialize Memory Buffer

Set DMA Source = I/O Device

Set DMA Destination = Memory Buffer

Set Transfer Size = Required Data Size

Enable DMA Controller

Enable DMA Completion Interrupt

Start DMA Transfer

WHILE DMA Transfer is not complete

 CPU performs other tasks

END WHILE

WHEN DMA Completion Interrupt occurs

 Save current CPU state

 Check DMA Status

```
IF transfer is successful THEN
    Set Transfer_Status = COMPLETE
    Process received data
ELSE
    Set Transfer_Status = ERROR
    Handle DMA error
END IF
```

```
Clear DMA Interrupt
Restore CPU state
Return to previous task
```

STOP

Key idea: The **DMA controller performs the actual data transfer**, so the CPU is interrupted only when the transfer is completed or an error occurs.

2. Vectored Interrupt Handling with Priority

Objective: Handle interrupts from multiple devices according to priority and dispatch the correct service routine.

PSEUDOCODE: PRIORITY_VECTORED_INTERRUPT_HANDLER

START

```
Initialize Interrupt Controller
Initialize Vector Table

Assign priority to each device
```

WHILE System is running

IF Interrupt Request is detected THEN

Identify interrupting device

Obtain interrupt vector

Determine device priority

IF higher-priority interrupt is active THEN

Save current CPU state

Suspend current service routine

END IF

Jump to corresponding Interrupt Service Routine (ISR)

Execute ISR

Clear device interrupt flag

Restore previously saved CPU state

Return from Interrupt

END IF

END WHILE

STOP

Key idea: The **vector directly identifies the required ISR**, avoiding unnecessary searching and reducing interrupt latency

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Objective: Select the appropriate I/O mechanism according to latency and bandwidth requirements.

PSEUDOCODE: DYNAMIC_IO_SELECTION

START

Read Device Requirements

Input:

Required_Latency

Required_Bandwidth

Device_Type

IF Required_Latency is HIGH THEN

Select MEMORY_MAPPED_IO

Map device registers into memory address space

Use normal memory instructions

Enable fast register access

ELSE IF Required_Bandwidth is HIGH THEN

Select MEMORY_MAPPED_IO

Map device registers into memory address space

Use efficient block data transfers

Enable DMA if required

ELSE

Select IO_MAPPED_IO

Assign separate I/O addresses

Use dedicated I/O instructions

END IF

Configure selected I/O mechanism

Test I/O communication

IF latency or bandwidth requirements are not satisfied THEN

Switch to alternative I/O mechanism

Reconfigure device

END IF

Continue normal I/O operation

STOP

Key idea: Memory-mapped I/O is generally preferred when fast register access and high-performance transfers are important, while **I/O-mapped I/O** can be used for simpler, lower-bandwidth peripherals.

4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt

Objective: Manage multiple devices while preventing starvation and ensuring fair access to the communication bus.

PSEUDOCODE: FAIR_BUS_ARBITRATION

START

Initialize PCIe Controller

Initialize USB Controller

Initialize Thunderbolt Controller

Create Device Request Queue

Assign priority to each device

WHILE System is running

 Check all devices for bus requests

 Add requesting devices to Request Queue

 IF Request Queue is not empty THEN

 Select device according to:

1. Priority
2. Waiting time
3. Round-robin order

IF device has waited for too long THEN

 Increase its priority

END IF

Grant bus access to selected device

Device performs data transfer

Monitor transfer

IF transfer is complete THEN

 Release bus

 Remove device from Request Queue

END IF

ELSE

 Keep bus in idle state

END IF

Check for high-priority requests

IF emergency/high-priority request occurs THEN

 Temporarily prioritize that request

END IF

END WHILE

STOP

Key idea: Priority + round-robin + aging provides fast service for important devices while preventing low-priority devices from being permanently starved.

5. Hybrid I/O Communication Mechanism

Objective: Use polling for low-speed devices and DMA with interrupts for high-speed devices.

PSEUDOCODE: HYBRID_IO_COMMUNICATION

START

Initialize all I/O devices

FOR each device

Determine device speed

IF device speed is LOW THEN

Configure device for POLLING

Add device to Polling List

ELSE

Configure DMA Controller

Set DMA Source = High-Speed Device

Set DMA Destination = Main Memory

Enable DMA Completion Interrupt

Add device to DMA List

END IF

END FOR

WHILE System is running

/* Handle low-speed devices */

FOR each device in Polling List

Check device status

IF data is available THEN

Read data from device

Store data in memory

END IF

END FOR

/* Handle high-speed devices */

FOR each device in DMA List

IF new data transfer is required THEN

 Configure DMA transfer

 Start DMA transfer

END IF

END FOR

/* Handle DMA interrupts */

IF DMA Interrupt occurs THEN

 Identify DMA device

 Check transfer status

 IF transfer is successful THEN

 Mark transfer as COMPLETE

 Process received data

 ELSE

 Handle DMA error

 END IF

 Clear DMA interrupt

END IF

CPU continues other tasks

END WHILE

STOP

Key idea: Polling is suitable for **low-speed devices**, while **DMA + interrupts** is better for high-speed devices because it greatly reduces CPU involvement.

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient